SchoolOS System Design Architecture

compass PHASE 1 — Foundation (Core Skills) **Technology Stack & Fundamentals**

🌐 **Web & Core Fundamentals**

- **Runtime**: Node.js (TypeScript) - Leveraging strict typing for robustness.
- **Frontend**: React 19 + Vite - utilizing modern hooks and concurrent features.
- **Styling**: Tailwind CSS + Radix UI - Utility-first styling with accessible primitives.
- **Protocol**: RESTful APIs via Express & WebSockets via Socket.io.

🧠 PHASE 2 — System Design Knowledge (HLD) **High-Level Architecture for SchoolOS**

Architecture Pattern: **Layered Hybrid Architecture**

1. **Client Layer**: React SPA (Single Page Application).
2. **Gateway Layer**: Express.js API Gateway handling routing & validation.
3. **Service Layer**: Modular services (PMS, Growth, Auth) isolating business logic.
4. **Data Layer**: Supabase (PostgreSQL) + Redis (Caching).

Core Design Principles:

- **Monolithic Backend (Initial)**: Single `server/src/app.ts` entry point for simplicity and shared type safety.
- **Modular Services**: Separation of concerns (e.g., `pms.service.ts`, `growth.service.ts`).

📐 PHASE 3 — Architecture Planning **SchoolOS Logic & Flows**

Functional Requirements:

- **Multi-Tenancy**: Hierarchical data (School -> Department -> Team).
- **RBAC+**: 5-level role system (SuperAdmin to Guest) with granular overrides.
- **Real-time Collaboration**: Live chat & task updates.
- **Educational Analytics**: Teacher performance & student growth tracking.

Non-Functional Requirements:

- **Data Isolation**: Strict separation between departments via RLS.
- **Performance**: <100ms response time for dashboard analytics.
- **Scalability**: Capable of supporting multiple school branches.

Data Flow Diagram:

```
[Client] -> [Express API] -> [Auth Middleware] -> [Service Layer] -> [Prisma ORM] ->
[PostgreSQL]
```

🎨 PHASE 4 — Frontend Development **Client-Side Architecture**

Tech Stack:

- **Framework**: React 19 (Vite)
- **State Management**: `React Context` (Auth) + `TanStack Query` (Server State).
- **Routing**: `react-router-dom` v7.

Folder Structure (`src/`):

- `modules/` : Feature-based (PMS, Auth, Growth).

- `components/ui/` : Reusable primitives (Buttons, Cards).
- `services/` : API wrappers (`api.js` , `socketService.js`).
- `hooks/` : Custom logic (`useAuth` , `useLocalStorage`).
- `context/` : Global state providers.

⚙️ PHASE 5 — Backend Development **Server-Side Architecture**

Tech Stack:

- **Server**: Express.js
- **Language**: TypeScript
- **Validation**: Zod (Runtime schema validation).

Backend Structure (`server/src/`):

- `controllers/` : Request handling & input sanitization.
- `services/` : Complex business logic & DB interactions.
- `routes/` : API endpoint definitions (`/api/v1/*`).
- `middleware/` : Auth checks & error handling.
- `prisma/` : Database schema & migrations.

🗄️ PHASE 6 — Database Architecture **Data Persistence Strategy**

Primary Database: **PostgreSQL** (Managed by Supabase)

- **ORM**: Prisma Client.
- **Why**: Structured relational data is crucial for school hierarchies.

Key Schema Entities:

- `School` & `Department` : Organizational Nodes.
- `Profile` : Users with extended metadata.
- `Project` & `Task` : Core PMS work units.
- `GrowthMetric` : Analytical data points.

Advanced Concepts:

- **Indexing**: On `email` , `departmentId` , and `schoolId` for fast lookups.
- **Relations**: Strict referential integrity (Cascading deletes for Projects).

🔐 PHASE 7 — Security Architecture **Trust & Safety Layer**

Authentication:

- **Mechanism**: JWT (JSON Web Tokens) via Supabase/Custom.
- **Context**: `AuthContext` provides user identity globally.

Security Practices:

- **RBAC Middleware**: Enforces role checks before controller execution.
- **Input Sanitization**: All inputs validated against Zod schemas.
- **Helmet**: Secure HTTP headers.
- **CORS**: Restricted cross-origin resource sharing.

🧪 PHASE 8 — Testing Strategy **Quality Assurance**

- **Unit Testing**: Testing individual utility functions and services.
- **Integration Testing**: Verifying flows between API and Database.

- **Manual Testing**: Postman collections for API validation.

☁️ PHASE 10 — Production Architecture **Live System Infrastructure**

Infrastructure:

- **Database**: Supabase (Cloud PostgreSQL).
- **Backend hosting**: Ready for Render/Railway (Containerized).
- **Frontend hosting**: Vercel/Netlify (Static edge).

Scaling Components:

- **Redis**: Caching session data and heavy analytics.
- **BullMQ**: Background job processing for report generation.

📈 PHASE 11 — Monitoring & Maintenance **Observability**

- **Logs**: Structured logging via Morgan/Winston.
- **Database Monitoring**: Supabase Dashboard (Query performance, Connection pool).
- **Health Checks**: `/health` endpoint for uptime monitoring.

🧩 PHASE 12 — Scalability (Advanced) **Future Growth Strategies**

- **Horizontal Scaling**: Stateless Express instances behind a Load Balancer.
- **Read Replicas**: Distributing analytics queries to read-only DB instances.
- **Code Splitting**: Dynamic imports in React to reduce initial bundle size.

⭐ PHASE 13 — Professional Practices **Standards & Ops**

- **Documentation**: Markdown guides in `docs/` (Architecture, Setup).
- **Linting/Formatting**: ESLint + Prettier configuration.
- **Strict TypeScript**: `noImplicitAny` enabled to prevent type errors.